PES University, Bengaluru

(Established under Karnataka Act 16 of 2013)

END SEMESTER ASSESSMENT (ESA) - DEC 2023

UE19CS351 - Compiler Design

Total Marks : 100.0

1.a. i) What is Compiler? How is it different from an Interpreter.
ii) Explain in brief (any two) types of compilers. (4.0 Marks)

1.b. Why is buffering used in lexical analysis? What are the commonly used buffering methods? (4.0 Marks)

1.c. With a neat diagram explain the design of a lexical analyzer. (4.0 Marks)

1.d. What are Lexical Errors? Write down the sequence of lexemes for the following code segment
for(i=0; i <= 10; i++) { printf(“%d”,i);} (4.0 Marks)

1.e. What is Lex and Yacc? Explain in brief. (4.0 Marks)

2.a. Explain in detail any two Error-Recovery Strategies. (4.0 Marks)

2.b. Consider the following grammar:
S → +SS
S → *SS
S → a

1. Calculate FIRST and FOLLOW
2. Construct predictive parsing table
3. Show how to parse the input *+aa*

(6.0 Marks)

2.c. Consider the following grammar:
S → SS+
S → SS*
S → a

1. Construct LR(0) automata
2. Construct SLR parsing table to check whether the given grammar is SLR or not.

(6.0 Marks)

2.d. With simple example explain the following terms (1 mark each):
i) Left-recursive grammar
ii) Leftmost derivation
iii) Ambiguous grammar
iv) LR(0) item (4.0 Marks)

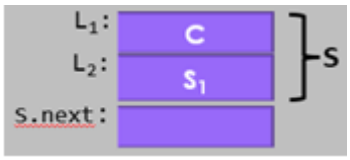
3.a. Explain the following in detail
i) Synthesized and Inherited Attributes
ii) L-attributed SDT and S-attributed SDT (8.0 Marks)

3.b. Consider the following grammar for variable declaration in C language:

$D \rightarrow T\ L$
$L \rightarrow L_1$, id | id
$T \rightarrow$ int | float

Write semantic rules to add/update type (i.e., int/float) information of the variables in declaration statement to the symbol table during parsing.
(4.0 Marks)

3.c. Write syntax directed definition (SDD) to generate intermediate code for while-loop.
Given: S → while (C) S_1



(4.0 Marks)

3.d. Write syntax directed definition (SDD) to generate three-address intermediate code for given Conditional Boolean expression production.
$C \rightarrow B_1 \&\& B_2$
Note: Use function *new()* that generates new labels. (4.0 Marks)

4.a. Write short notes on:
i) Triples
ii) Static Single Assignment (SSA) (5.0 Marks)

4.b. Construct control-flow graph (CFG) for the following three-address code.
(5.0 Marks)

```
   x = n
   y = m
   ifFalse x < y goto L1
   t1 = x + 1
   x = t1
   t2 = y - 1
   y = t2
   goto L2
L1:
   y = x + 2;

L2:
   z = x * y;
   return z;
```

4.c. Optimize the code below by applying the following code transformations: Constant Folding, Constant Propagation and dead code elimination.

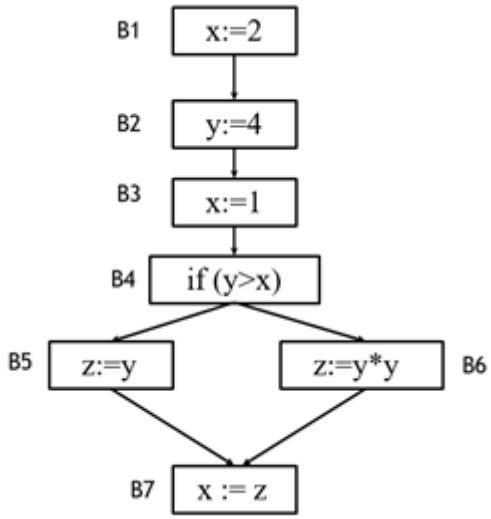
(5.0 Marks)

```
int a = 30;
int b = 9 - (a / 5);
int c;
c = b * 4;

if (c > 10) {
c = c - 10;
}
return c * (60 / a);
```

4.d. Calculate use[B], def[B] and then find IN[B], OUT[B] by applying *Live-variable Analysis Algorithm* on the following flow graph.

(5.0 Marks)



5.a. Explain in brief any four issues in the design of a code generator.

(4.0 Marks)

5.b. Explain the following with a neat diagram
i) Activation record
ii) Activation tree (6.0 Marks)

5.c. Explain in brief the different ways in which a procedure can access nonlocal data on a runtime Stack. (4.0 Marks)

5.d. Generate target code sequence for the following basic block three address statements with register and address descriptors.

```
1.  x:= y-z
2.  z:= x*x
3.  y:= z
4.  x:= y-z
```

Note 1: Assuming only two registers (R1 and R2) are available and all the variables are live on exit from the basic block. No temporary variables are used.
Note 2: Assuming that all arithmetic operations take their operands from registers. (6.0 Marks)